9. Gyakorlat

Algoritmusok és adatszerkezetek 2.

Minimális feszítőfák

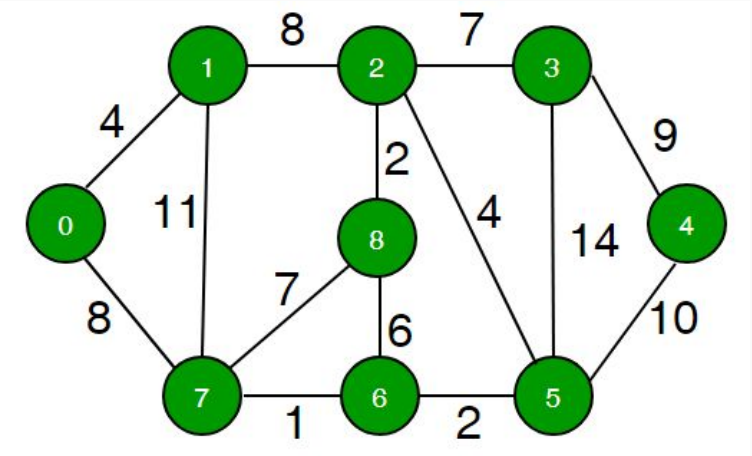
Ismétlés / folytatás

Kruskal algoritmus

1. A halmaz üres kezdőértéket kap, és létrejön a gráf csúcsait külön-külön tartalmazó |V| darab fa.
2. Egy ciklussal megyünk, amíg van feldolgozatlan él. Kivesszük a következő legkisebb súlyú élt.
3. Minden így kapott (u, v) élre meg kell vizsgálni, hogy a végpontjaik ugyanahhoz a fához tartoznak-e.
    a. Ha igen, akkor nem kell az éllel foglalkozni (az erdőben kör alakulna ki).
    b. Ha nem, akkor (u, v) élt hozzá kell venni A-hoz, és a két fa csúcsait összevonjuk. (Ebben az esetben a két csúcs különböző fákhoz tartozott.

5. Feladat

Szemléltessük a Kruskal algoritmus működését az alábbi gráfon:



Unió holvan adatszerkezet

Az "unió-holvan" adatszerkezet a következő probléma megoldására született: adott véges sok halmaz, amelyeken a következő lépéseket kell - előre nem ismert sorrendben, sokszor egymás után - elvégezni:

- megkérdezhetjük két elemről, hogy azonos halmazban vannak-e;
- egyesíthetünk két halmazt

Vázlatosan a következőt csináljuk: a halmazokat (felfelé) irányított fákkal reprezentáljuk. Minden elemnek lesz egy "szülő-pointere", a fa gyökerének önmaga. Egy elem halmazát a fája gyökerével azonosítjuk. Az apa-pointerek követésével ez gyorsan meghatározható. Két halmaz egyesítésekor az egyik gyökerének apa-pointerét a másik gyökerére állítjuk.

6. Feladat

Adjuk meg a Kruskal algoritmust, használva az Unió holvan adatszerkezetet!

Absztrakt leírás

i := 0; MinFeszFa := {}

Ciklus az élek élsúly szerint növekvően rendezett listáján amíg i < n - 1

    e := a soron következő él

    Ha MinFeszFa+{e} nem tartalmaz kört akkor

        MinFeszFa += {e}

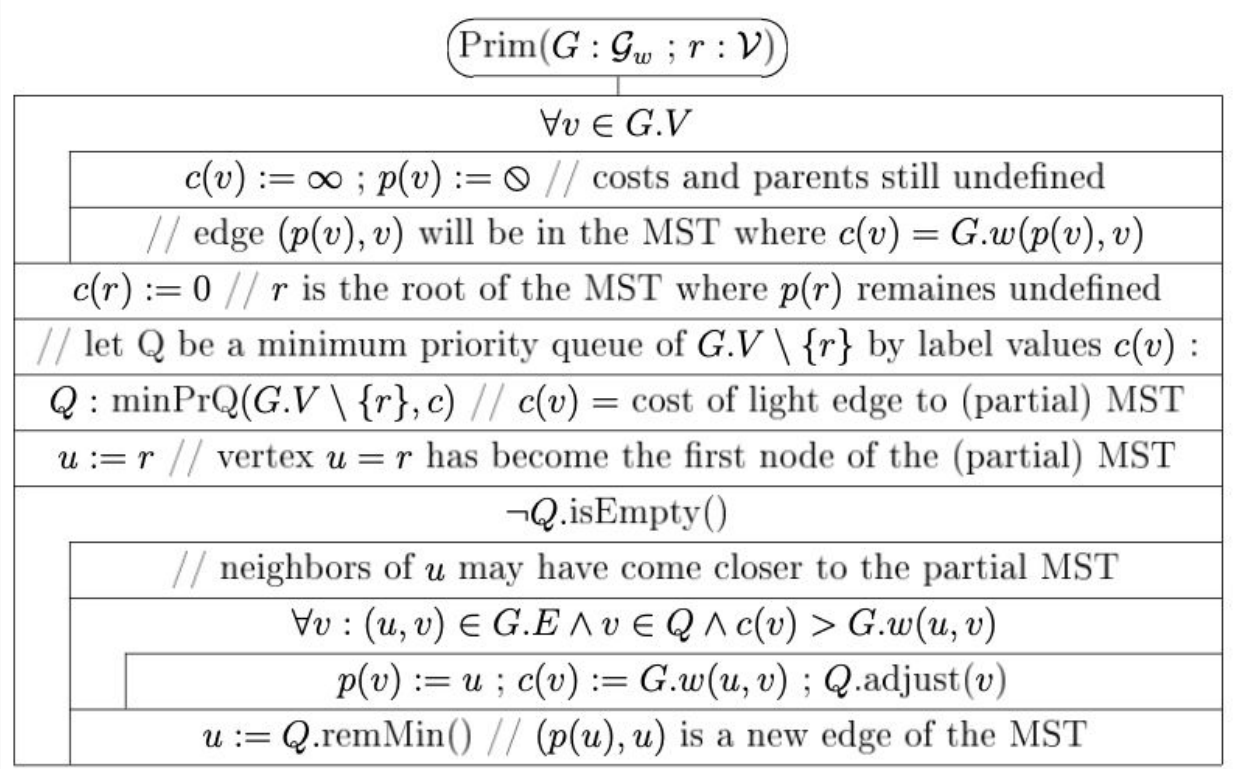
        i++

Prim algoritmusa

Az A halmaz egyetlen fa. Az A-hoz hozzáadott biztonságos él mindig a legkisebb súlyú olyan él, amelyik egy A-beli és egy A-n kívüli csúcsot köt össze. Mivel mohó algoritmus, így minden lépésben olyan éllel bővíti a fát, amely a lehető legkisebb mértékben növeli a fa teljes súlyát.

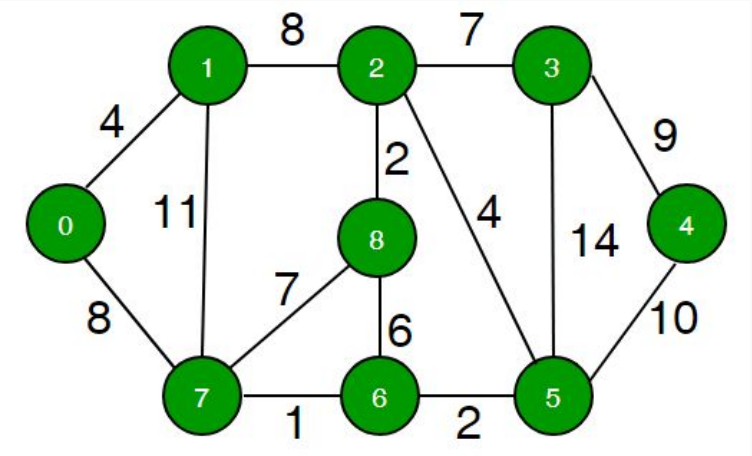
Minden lépésben azt a könnyű élt vesszük hozzá az A fához, amelyik egy A-beli csúcsot köt össze a GA = (V, E) egy izolált csúcsával.

Prim algoritmusa



6. Feladat

Szemléltessük a Prim algoritmus működését az alábbi gráfon:



Minimális összköltségű utak

Dijkstra algoritmusa

Definíciók ismétlése

Út: $u_1, u_2, \ldots u_k$, úgy, hogy $j=1..k-1$: $(u_j,u_{j+1}) \in E$

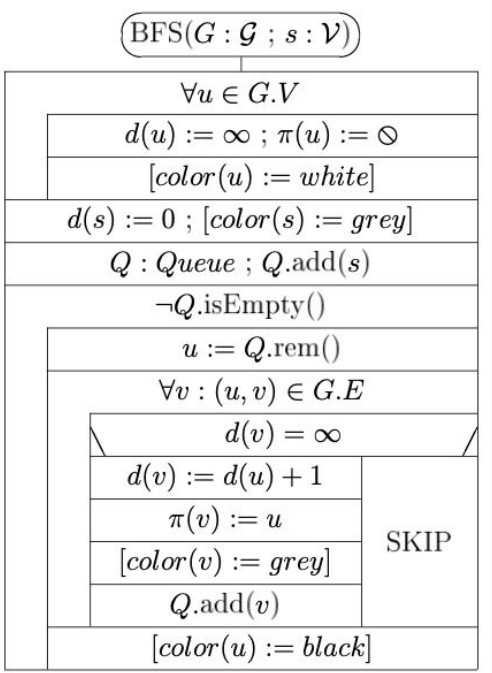
Összköltsége súlyozott gráfban: $\Sigma_{j=1..k-1} w(u_j,u_{j+1})$

Minimális összköltségű út u,v között: az összes út közül a minimális összköltségű.

Mikor nincs minimális összköltségű út?

Szélességi bejárás átismétlése

Adjunk élsúlyokat a gráfhoz és nézzük meg, hogy hol kellene módosítani a BFS algoritmust, ha azt szeretnénk, hogy az élsúlyokat is figyelembe vegye?

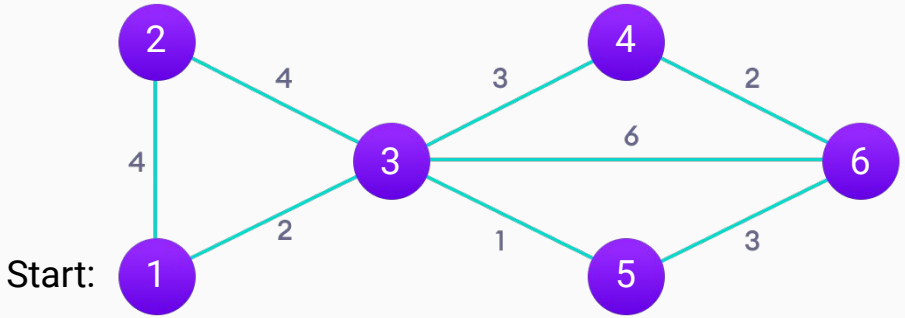


Mohó stratégia

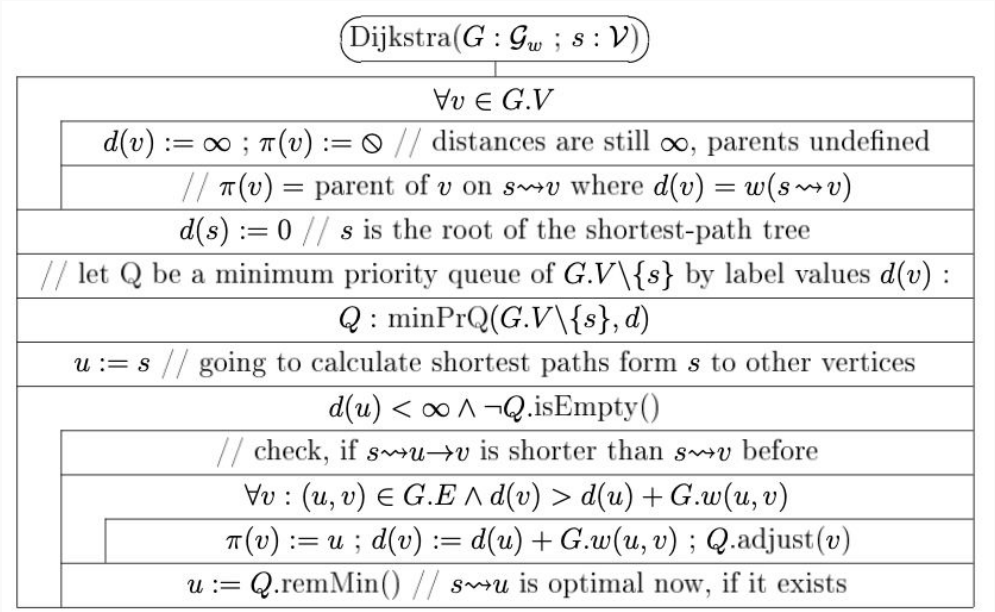
Mohó algoritmusok: olyan heurisztikára épülő algoritmusok, amelyek a lokálisan legjobb választási lehetőségekre építenek.

Hogyan alkalmazhatjuk a mohó stratégiát a legalacsonyabb összköltségű utaknál a BFS algoritmus kiegészítésére?

Példa



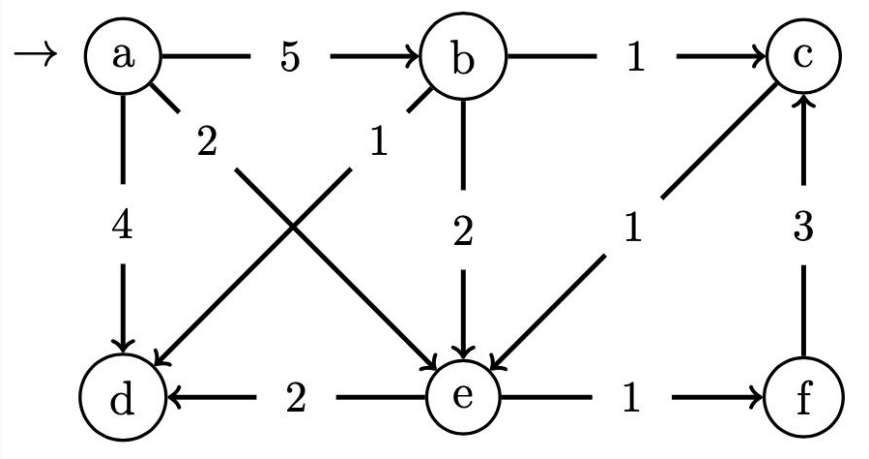
Dijkstra algoritmusa



1. Feladat

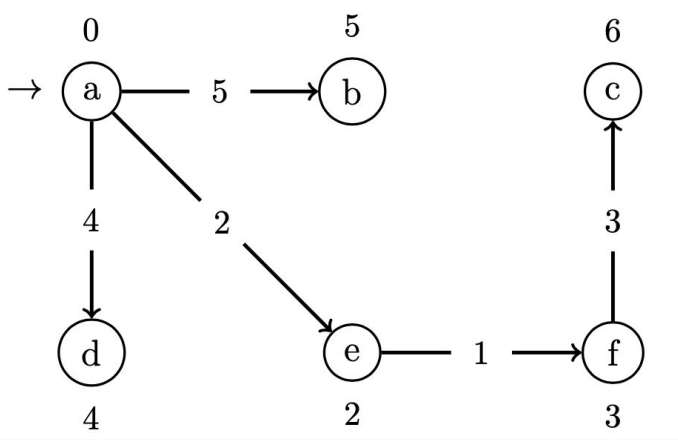
Szemléltessük a Dijkstra algoritmus működését az alábbi gráfon. Minden lépésben adjuk meg minden ***v*** csúcsra a ***d(v)*** és ***π(v)*** értékeket, valamint az aktuálisan kiterjesztett csúcsot.

Start csúcs: ***a***



Megoldás

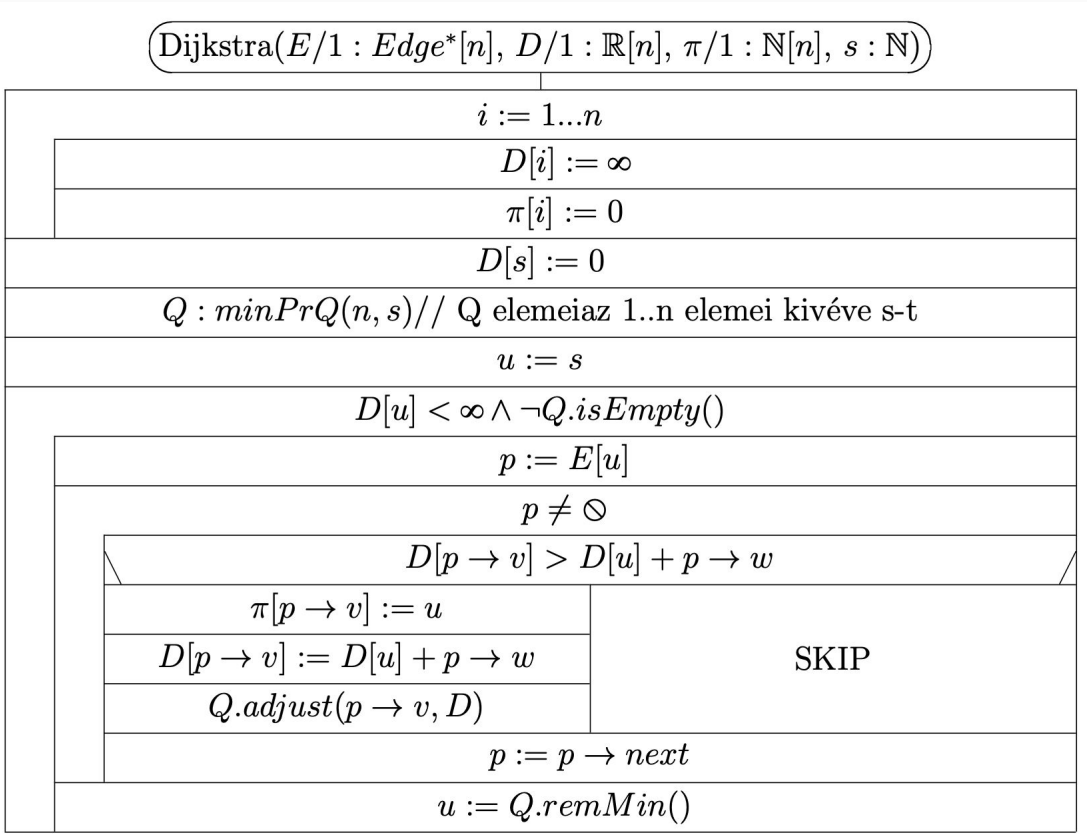
d értékek Q-ban						kiterjesztett csúcs:d	π címkék változásai					
a	b	c	d	e	f		a	b	c	d	e	f
0	∞	∞	∞	∞	∞	–	∅	∅	∅	∅	∅	∅
	5	∞	4	2	∞	a : 0		a		a	a	
	5	∞	4		3	e : 2						e
	5	6	4			f : 3			f			
	5	6				d : 4						
		6				b : 5						
						c : 6						
0	5	6	4	2	3	eredmény	∅	a	f	a	a	e



2. Feladat

Készítsük el Dijkstra algoritmusát reprezentációs szinten szomszédsági listás ábrázolás esetén. Az algoritmusban tegyük fel, hogy a gráf csúcsait 1-től n-ig sorszámozzuk. A paraméterek pontos meghatározása a feladat része.

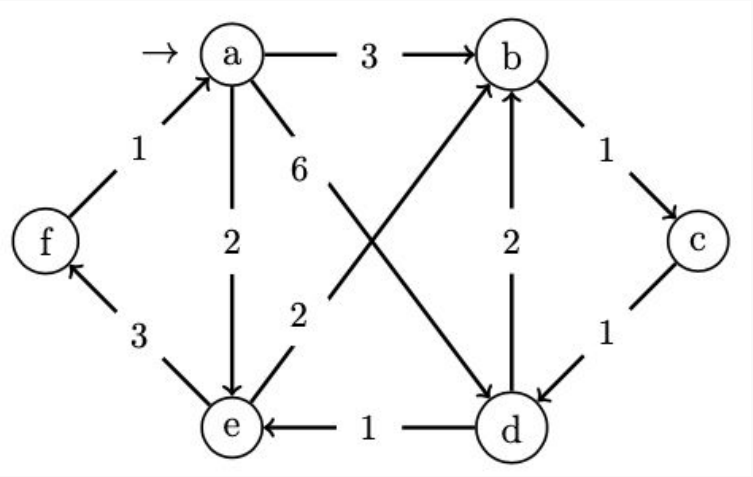
Megoldás


Dijkstra($E/1 : Edge^*[n]$, $D/1 : \mathbb{R}[n]$, $\pi/1 : \mathbb{N}[n]$, $s : \mathbb{N}$)
$i := 1...n$
$D[i] := \infty$
$\pi[i] := 0$
$D[s] := 0$
$Q : minPrQ(n, s)$// Q elemeiaz 1..n elemei kivéve s-t
$u := s$
$D[u] < \infty \wedge \neg Q.isEmpty()$
$p := E[u]$
$p \neq \oslash$
$D[p \to v] > D[u] + p \to w$
$\pi[p \to v] := u$
$D[p \to v] := D[u] + p \to w$
$Q.adjust(p \to v, D)$
SKIP
$p := p \to next$
$u := Q.remMin()$

3. Feladat

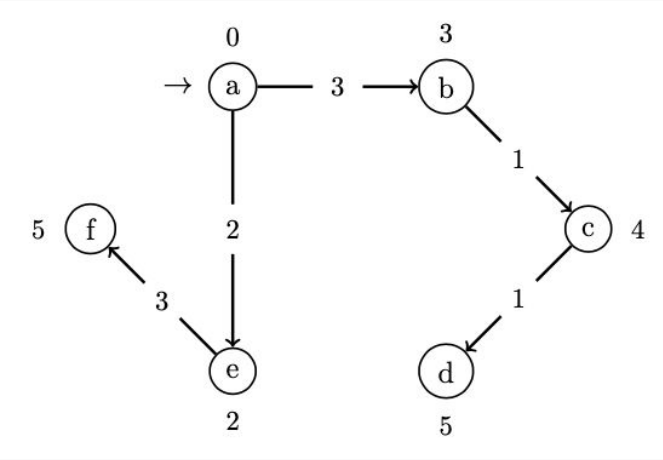
Szemléltessük a Dijkstra algoritmus működését az alábbi gráfon. Minden lépésben adjuk meg minden ***v*** csúcsra a ***d(v)*** és ***π(v)*** értékeket, valamint az aktuálisan kiterjesztett csúcsot.

Start csúcs: ***a***



Megoldás

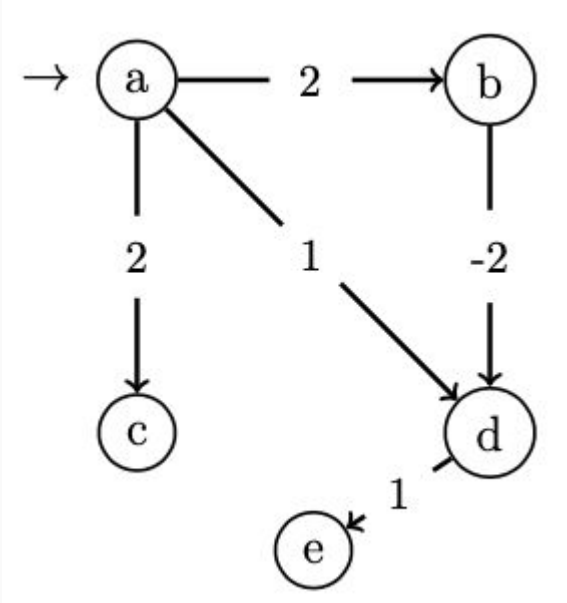
d értékek Q-ban						kiterjesztett csúcs:d	π címkék változásai					
a	b	c	d	e	f		a	b	c	d	e	f
0	∞	∞	∞	∞	∞	–	∅	∅	∅	∅	∅	∅
	3	∞	6	2	∞	a : 0		a		a	a	
	3	∞	6		5	e : 2						e
		4	6		5	b : 3			b			
			5		5	c : 4				c		
					5	d : 5						
0	3	4	5	2	5	eredmény	∅	a	b	c	a	e



4. Feladat

Vizsgáljuk meg Dijkstra algoritmusá abban az esetben, ha negatív élsúlyok is megengedettek. Javítható-e az algoritmus, hogyha minden élsúlyt megnövelünk annyival, hogy ne legyen negatív élsúly? Ha igen, miért, ha nem, miért?

Problémát okoz-e a Dijkstra algoritmusban a 0 élsúly? Miért?



5. Feladat

Adott $U = \{V_1, V_2, \ldots V_n\}$ halmaz, amely városokat reprezentál. Minden városhoz adott egy adott napi légiforgalmi menetrend az alábbi formában:

Menetrend_i = {[célállomás, indulás időpontja, érkezés időpontja, repülőjegy ára],...}

Adjunk eljárást, amely megadja A városból elindulva menetrend szerinti járatokkal hogyan lehet napon belül a **legolcsóbban** megérkezni B városba, figyelmen kívül hagyva az utazás időtartamát, de figyelembe véve az utazás megvalósíthatóságát (azaz pl. nem indulhatunk korábban, mint ahogyan megérkezünk valahova). A,B eleme U.

6. Feladat

Adott U = {V_1, V_2, … V_n} halmaz, amely városokat reprezentál. Minden városhoz adott egy adott napi légiforgalmi menetrend az alábbi formában:

Menetrend_i = {[célállomás, indulás időpontja, érkezés időpontja, repülőjegy ára],...}

Adjunk eljárást, amely megadja, hogy egy kitűzött időpontban – egy adott napon belül - A városból elindulva menetrend szerinti járatokkal hogyan lehet a **leggyorsabban** megérkezni B városba, figyelmen kívül hagyva az utazás árát, de figyelembe véve az utazás megvalósíthatóságát (azaz pl. nem indulhatunk korábban, mint ahogyan megérkezünk valahova). A, B eleme U.

7. Feladat

Legyen G súlyozott gráf $c: E \to \{0, 1\}$ súlyfüggvénnyel. Valósítsuk meg a Dijkstra-algoritmust a szélességi bejárás módosításával, amelyben egy kétvégű sort használunk (azaz a sor mindkét végére helyezhetünk be új elemet, és vehetünk ki elemet).

8. Feladat

Adott egy irányított hálózat, amelynek minden éléhez hozzárendelünk egy nemnegatív kapacitásértéket. Adjunk algoritmust két pont közötti legszélesebb (legnagyobb kapacitású) út megkeresésére. Egy út kapacitása alatt az utat alkotó élek kapacitásának minimumát (a legszűkebb keresztmetszetet) értjük.

Ötlet: gondoljunk arra, hogy a Dijsktra egy mohó algoritmus. Hogyan tudjuk ezt a stratégiát ebben az esetben alkalmazni?

9. Feladat

Egy kommunikációs hálózatot egy irányítatlan gráffal modellezünk, ahol $(u, v) \in E$ él, az u-t és a v-t összekötő kommunikációs csatornát szimbolizálja. Az élekhez hozzárendeltük annak valószínűségét, hogy a csatorna működik (nem sérült). Az egyes csatornák működőképessége egymástól független. Adjunk algoritmust két pont közötti legmegbízhatóbb út előállítására.